

COSC2673_2610 Computational Machine Learning

Assessment 1 – Airbnb Price Prediction Using Regression Models

Student: Alissa Nguyen (s4085541)

1. Introduction

In this task, I develop a variety of regression models to make prediction on Airbnb listing price based on a real-world dataset. The problem is considered as a regression supervised learning problem. Since the target variable of interest is 'price', the value of which is continuous, we want to see how the models perform on the data set and how well they could generalise. Three types of regression models including Linear Regression, Ridge Regression, Lasso Regression are built and compared in terms of evaluation criteria like RMSE and the R^2 coefficient of determination.

Before running the model, ensure that:

1. A Python environment is (Anaconda or standard Python with Jupyter Notebook)
2. Install the following libraries:
 - pandas
 - numpy
 - matplotlib
 - scikit-learn
3. Download the data files (train_data.csv and test_data.csv) into your working directory.

To run this notebook:

4. Start jupyter notebook/lab
5. Run the cells from top to bottom order
6. The notebook will automatically do the following:
 - data loading and preprocessing
 - models training and evaluating
 - model comparison
 - choose the optimal model
 - generate predictions on the test data
7. The predicted data will be automatically saved as csv file called “s4085541_predictions.csv”

2. Dataset and Features

train_df.info()	test_df.info()
<pre><class 'pandas.core.frame.DataFrame'> RangeIndex: 8586 entries, 0 to 8585 Data columns (total 16 columns): # Column Non-Null Count Dtype --- --- 0 host_is_superhost 8586 non-null object 1 city 8586 non-null object 2 country 8586 non-null object 3 latitude 8586 non-null float64 4 longitude 8586 non-null float64 5 room_type 8586 non-null object 6 accommodates 8586 non-null int64 7 bathrooms 8586 non-null float64 8 bedrooms 8586 non-null float64 9 beds 8586 non-null float64 10 price 8586 non-null int64 11 minimum_nights 8586 non-null int64 12 number_of_reviews 8586 non-null int64 13 review_scores_rating 8586 non-null float64 14 instant_bookable 8586 non-null object 15 calculated_host_listings_count 8586 non-null int64 dtypes: float64(6), int64(5), object(5) memory usage: 1.0+ MB</pre>	<pre><class 'pandas.core.frame.DataFrame'> RangeIndex: 8585 entries, 0 to 8584 Data columns (total 15 columns): # Column Non-Null Count Dtype --- --- 0 host_is_superhost 8585 non-null object 1 city 8585 non-null object 2 country 8585 non-null object 3 latitude 8585 non-null float64 4 longitude 8585 non-null float64 5 room_type 8585 non-null object 6 accommodates 8585 non-null int64 7 bathrooms 8585 non-null float64 8 bedrooms 8585 non-null float64 9 beds 8585 non-null float64 10 minimum_nights 8585 non-null int64 11 number_of_reviews 8585 non-null int64 12 review_scores_rating 8585 non-null float64 13 instant_bookable 8585 non-null object 14 calculated_host_listings_count 8585 non-null int64 dtypes: float64(6), int64(4), object(5) memory usage: 1006.2+ KB</pre>

The training data has 8,586 rows and 16 columns, including the target variable (price), while the test set has the same number of rows but does not include the target.

The dataset includes Airbnb listings with a mix of numerical and categorical features.

Dataset	Rows	Columns
Train_data.csv	8586	16
Test_data.csv	8585	15

Train_data.csv has the target variable (price), while the test_data.csv does not.

Numerical features:

Categorical features:

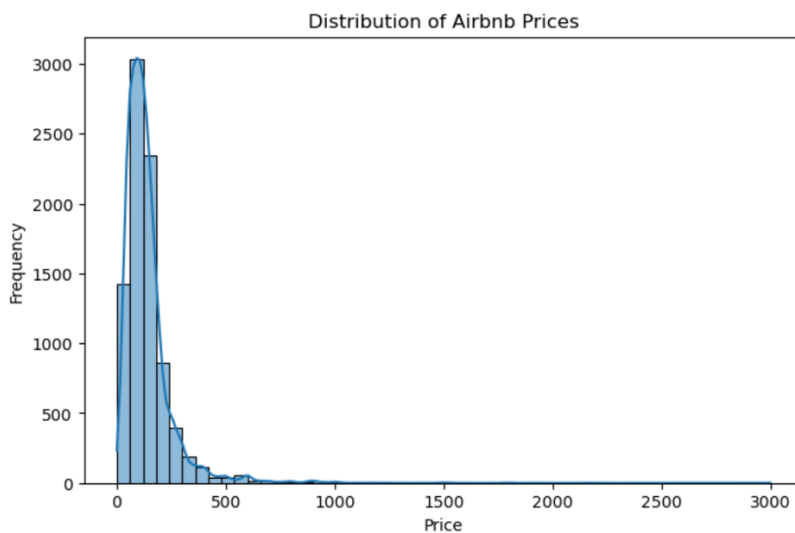
- latitude, longitude
- accommodates, bathrooms, bedrooms, beds
- minimum_nights, number_of_reviews
- review_scores_rating, calculated_host_listings_count
- host_is_superhost
- city, country
- room_type
- instant_bookable

One thing I noticed early on is that the dataset is quite clean, with no obvious missing values. However, I still kept the preprocessing steps in place because that was part of the workflow taught in class and makes the pipeline more robust. Another thing that stood out is that the numerical features are on very different scales, which is why scaling was necessary later on.

3. Data Analysis

After inspecting the dataset, I explored the data visually to get a better sense of how the features relate to price.

3.1 Price Distribution

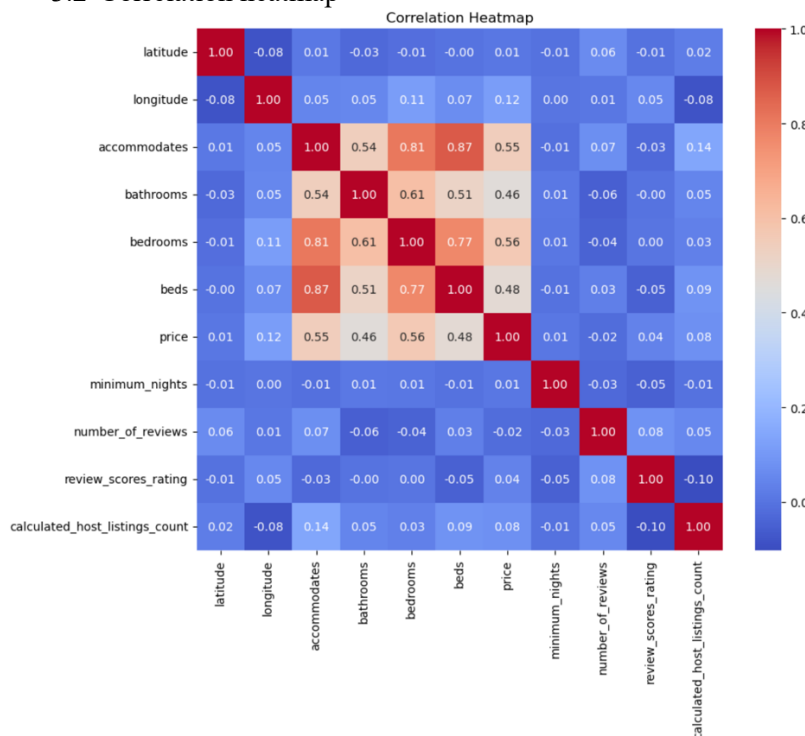


The distribution of Airbnb prices is right skewed. This means that, most listings are low to moderately price. A small number of listings are very expensive.

This skewness indicates that extreme values (outliers) exist. Linear models may struggle to accurately predict more high priced listings. Prediction errors are likely to increase for expensive properties.

Extreme prices may be harder for linear models to predict accurately.

3.2 Correlation heatmap



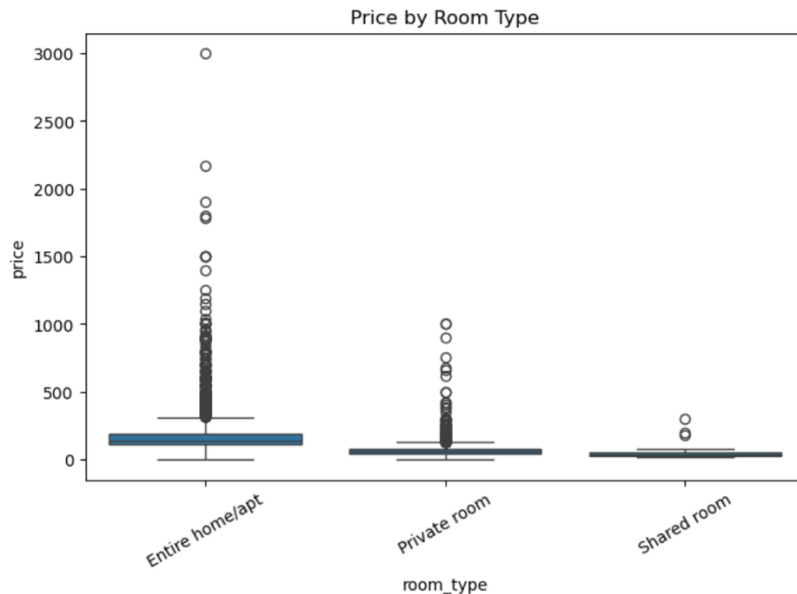
The heatmap shows pairwise correlations between numerical features.

- accommodates, bedrooms, and beds show moderate correlation with price (~ 0.5 – 0.56)
- Most other features have weak correlation with price
- Strong correlations exist between some predictors (accommodates $\Leftrightarrow$ beds)

Interpretation:

- No single feature strongly predicts price $\Rightarrow$ limits model performance
- Some multicollinearity exists $\Rightarrow$ justifies using Ridge/Lasso
- Supports why R^2 remains around ~ 0.39

3.3 Room type vs Price

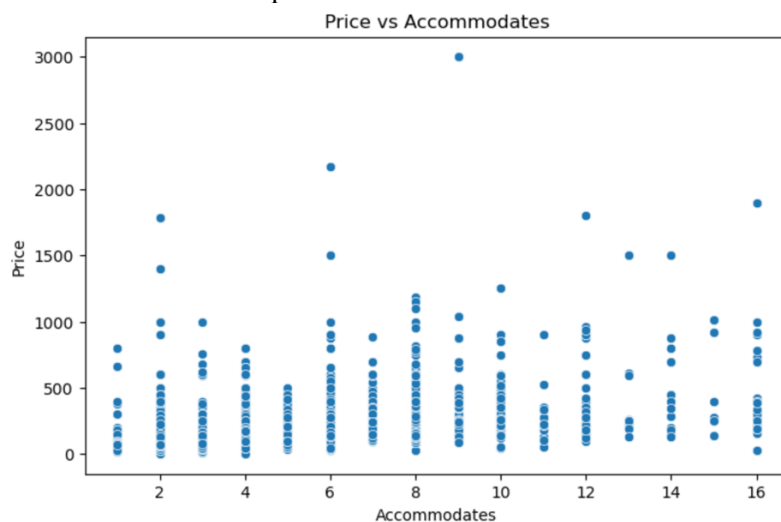


The boxplot clearly shows that entire home/apartment listings are significantly more expensive than the others. Private rooms appear to be cheaper, and shared rooms are cheapest.

Interpretation:

- room_type is an important predictor.
- Justifies the use of One-Hot Encoding in preprocessing.
- Shows categorical variables contribute meaningful signal.

4.4 Accommodates vs price



The scatter plot shows a general upward trend (more guests => higher price). However, the relationship is messy and dispersed.

Interpretation:

- The relationship is not strictly linear.
- Indicates that linear models will only partially capture this trend.
- Suggests potential benefit of non-linear models (not implemented here).

4. Preprocessing and split methods

```
X = train_df.drop(columns="price")
y = train_df["price"]

X_test_final = test_df.copy()

numeric_features = [
    "latitude",
    "longitude",
    "accommodates",
    "bathrooms",
    "bedrooms",
    "beds",
    "minimum_nights",
    "number_of_reviews",
    "review_scores_rating",
    "calculated_host_listings_count"
]

categorical_features = [
    "host_is_superhost",
    "city",
    "country",
    "room_type",
    "instant_bookable"
]
```

Before training the models, the dataset was prepared by separating the input features and the target variable. In the notebook, this was implemented by assigning all predictor variables to X and the target variable (price) to y. The test dataset was kept separate to ensure that it remained completely unseen during model training.

A preprocessing pipeline was constructed using scikit-learn's ColumnTransformer and Pipeline. For numerical features, median imputation was applied to handle potential missing values, followed by standardisation using StandardScaler to ensure all features are on a comparable scale. For categorical features, missing values were filled using the most frequent

category, and one-hot encoding was used to convert them into numerical form. Embedding these steps within a pipeline ensures that preprocessing is applied consistently during both training and prediction and helps prevent data leakage.

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])
```

The training data was then split into training and validation sets using an 80/20 split. This allows model performance to be evaluated on unseen data, providing a more realistic estimate of how the model will perform in practice. A fixed random state was used to ensure reproducibility of results.

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

5. Model implementation

Three regression models were implemented in the notebook: Linear Regression, Ridge Regression, and Lasso Regression. Linear Regression was used as a baseline model due to its simplicity and interpretability. Ridge and Lasso regression were included to address potential issues such as multicollinearity and overfitting through regularisation.

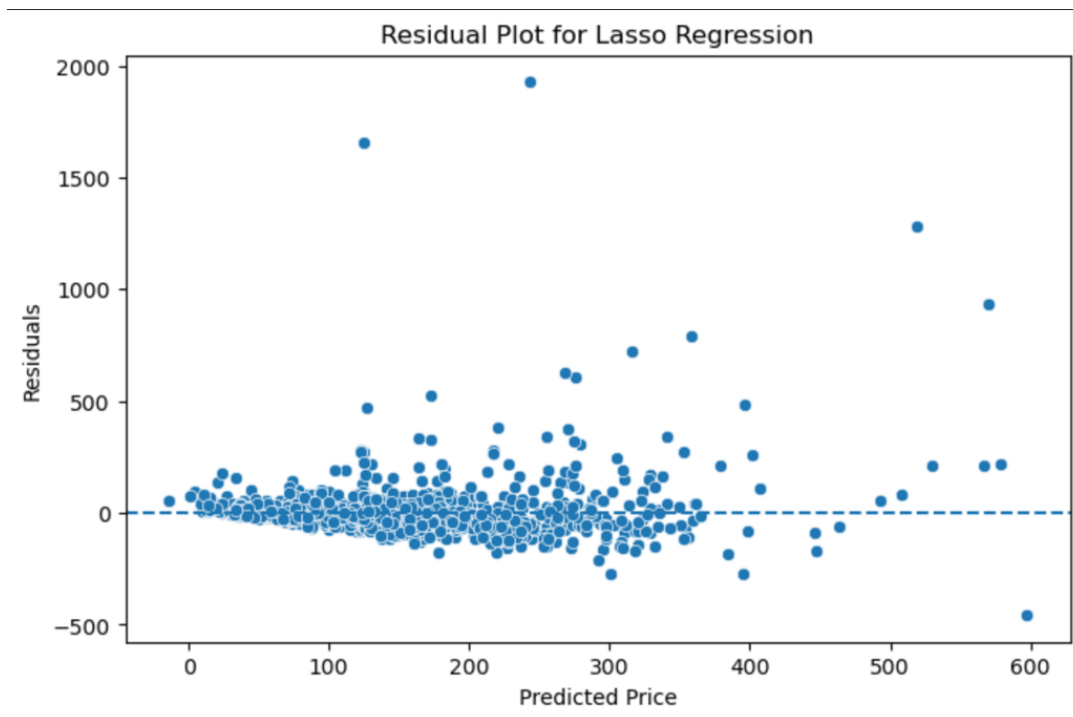
The results show that all three models perform similarly, with only minor differences in both RMSE and R^2 values. The validation R^2 scores are approximately 0.39 across all models, indicating moderate predictive performance. Lasso Regression achieves the highest validation R^2 , although the improvement is marginal. The validation RMSE values are consistently higher than the training RMSE values, suggesting that the models exhibit slight overfitting.

```
results_df = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression"],
    "Train RMSE": [train_rmse_lr, train_rmse_ridge, train_rmse_lasso],
    "Validation RMSE": [val_rmse_lr, val_rmse_ridge, val_rmse_lasso],
    "Train R2": [train_r2_lr, train_r2_ridge, train_r2_lasso],
    "Validation R2": [val_r2_lr, val_r2_ridge, val_r2_lasso]
})
```

results_df

	Model	Train RMSE	Validation RMSE	Train R2	Validation R2
0	Linear Regression	90.079454	103.683616	0.442426	0.395486
1	Ridge Regression	90.082739	103.668752	0.442385	0.395659
2	Lasso Regression	90.245972	103.562264	0.440363	0.396900

Further insight into model performance was obtained through diagnostic plots generated in the notebook. The residual plot shows that prediction errors are generally centred around zero, but the spread increases for higher predicted prices. This indicates that the model struggles more with expensive listings. Similarly, the actual versus predicted plot shows that predictions are more accurate for lower-priced listings, while higher-priced listings tend to be underestimated. These observations are consistent with the skewed price distribution identified during EDA.

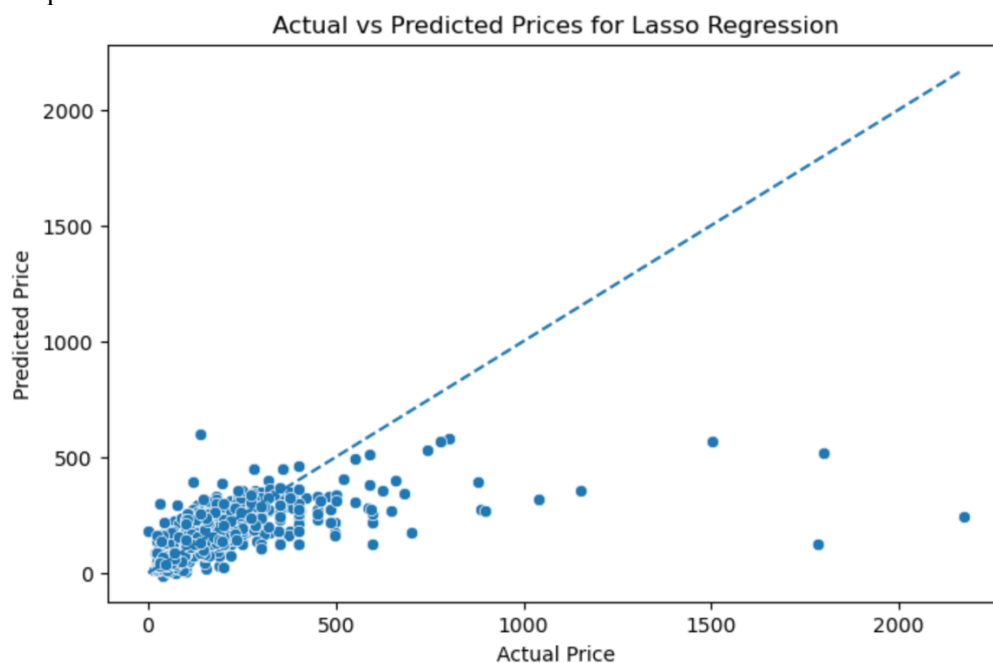


6. Models Selection

Lasso Regression was selected as the final model based on its slightly higher validation R^2 compared to the other models. In addition to its marginally better performance, Lasso has the advantage of performing feature selection by shrinking less important coefficients towards zero. However, given the minimal difference between models, this selection is primarily justified by performance rather than a significant structural advantage.

7. Final Model and Predictions

The selected Lasso model was retrained on the full training dataset to maximise the use of available labelled data. The trained model was then applied to the test dataset to generate predictions for Airbnb prices. These predictions were saved as a CSV file (s4085541_predictions.csv) in the required submission format. This step completes the end-to-end machine learning pipeline implemented in the notebook.



```
y_test_pred = final_model.predict(X_test_final)

pred_df = pd.DataFrame({"price": y_test_pred})
pred_df.to_csv("s4085541_predictions.csv", index=False)
pred_df.head()
```

	price
0	24.891241
1	128.745471
2	64.283836
3	18.070420
4	84.188302

8. Rationale

The approach I used in the notebook follows the general workflow that we covered in class, so I tried to keep things consistent rather than overcomplicating it. I started with basic inspection and EDA to understand what the dataset looks like, then moved on to preprocessing before training the models. Using a pipeline for preprocessing was important because it keeps everything consistent and avoids mistakes like applying transformations differently to training and validation data. It also made the code easier to manage since everything is handled in one place. I also chose to use a train-validation split instead of just training on all the data straight away. This made it easier to compare models fairly and see how they perform on unseen data. Even though the models themselves are quite simple, this setup helped make the evaluation more reliable.

9. Critical Analysis

Looking at the results overall, the models perform okay but not particularly well. The validation R^2 values are around 0.39, which means that a large part of the variation in price is still not being explained. From what I saw earlier in data analysis part, this is not too surprising. The correlations between features and price were only moderate, and the relationships were quite noisy. That already suggested that the models would have limited predictive power. Another thing that became clear from the evaluation plots is that the model struggles with higher-priced listings. In the actual vs predicted plot, a lot of the expensive listings are underestimated, and the residual plot also shows larger errors in that range. This is likely because those listings are less common, so the model doesn't learn them as well. A bigger limitation is probably the dataset itself. Important factors that affect price, like exact location quality, amenities, or demand at different times, are not included. Because of this, even a well-tuned model would still have difficulty making accurate predictions. Overall, the issue is not just the model choice, but the fact that the problem is more complex than what the available features can fully capture.

10. Improvements

There are a few ways the model could likely be improved based on what was observed. One obvious issue is the skewness of the price variable. Applying a log transformation would probably make the distribution more balanced and easier for a linear model to learn from. I chose not to implement this in the notebook since it was not explicitly covered in the earlier weeks, but it would be a reasonable next step. Another area for improvement is feature engineering. At the moment, the model relies on the existing features, but combining variables. For example, bedrooms relative to accommodates or creating location-based groupings could potentially capture more useful patterns. Finally, using more flexible models would likely improve performance. The current models are all linear, which limits their ability to capture more complex relationships in the data. Models like decision trees or ensemble methods could handle this better, although they are outside the scope of this assignment.

11. Ethics

Although this project is mainly technical, there are still some ethical considerations to keep in mind. If a model like this were used in practice, it could influence how prices are set for Airbnb listings. Because the model is not perfectly accurate and tends to underestimate higher-priced properties, relying on it too heavily could lead to poor decisions. There is also a risk that patterns in the data could reflect underlying biases, such as differences between locations, which the model would then reproduce. For this reason, it is important to treat the model as a support tool rather than something that should be blindly trusted. It is also important to be transparent about its limitations and ensure that the data is used responsibly.

12. Conclusions

Overall, this project shows that simple regression models can capture some general trends in Airbnb pricing, but their performance is limited. The validation R^2 values are around 0.39, which suggests that a large portion of the variation in prices is not explained by the model. Looking back at the earlier analysis, this result makes sense. The relationships between features and price are not very strong, and the distribution of prices is quite uneven. These factors make the problem harder than it might initially seem. While the models used here provide a reasonable starting point, improving performance would likely require better features or more advanced modelling approaches. Even so, the process followed in the notebook reflects a solid machine learning workflow and provides a good foundation for further improvement.

13. References

13.1 All figures and visualisations used in this report document are screenshots from my Jupyter notebook and are generated in the notebook as a part of the analysis.

13.2 Scikit-learn Documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.compose.TransformedTargetRegressor.html#sklearn.compose.TransformedTargetRegressor> (for implementing preprocessing pipelines and regression models).

13.3 COSC2673 Computational Machine Learning materials including lecture slides and labs.

13.4 Github repository consulted as a general reference and guide for understanding similar approaches to price prediction for Airbnbs: <https://github.com/DivyaSharma0795/Airbnb-Price-Prediction>

13.5 Reference for plots: <https://exploration.stat.illinois.edu/learn/Linear-Regression/Airbnb-Research-Goal-Conclusion/>

https://www.preprints.org/frontend/manuscript/a0779ebb883f0368cdcd80532900fab0/download_pub